A | Dart Appendix

A.1 The I/O library

The "dart:io" library is used to work with files, directories, WebSockets, HTTP clients and processes. Almost any I/O operation in Dart is executed asynchronously to avoid blocking the application so results are returned either via Future<T> or Stream<T>.

A.1.1 Files

In Dart you can read a file in two ways: all at once or lazily. Reading a file entirely requires having enough memory. Note that Uint8List is basically a List<int> so its contents can be traversed regularly like if they were items of a list.

```
final myFile = File('data.json');

// The entire content of 'data.txt' is read
final String content = await myFile.readAsString();

// Each line of the files is represented by a String
final List<String> contents = await myFile.readAsLines();

// Reads the file as a sequence of bytes (raw bytes)
final Uint8List data = await myFile.readAsBytes();
```

Consider using the above approaches when the file is not too big because it's entirely loaded in memory. In mobile apps you generally don't store big text files so calling File.readAsString() in Flutter is not a problem. With larger files, you probably should lazily read the contents using a stream.

```
final myFile = File('data.json');
```

```
final Stream<List<int>> stream = myFile.openRead();

final fileLines = utf8.decoder
    .bind(stream)
    .transform(LineSplitter());

try {
    await for (var line in fileLines) {...}
} catch (e) {
    print(e);
}
```

The file is closed as soon as the `await for` terminates. The `utf8` variable is a constant part of the `dart:convert` library. Writing to a file can easily be done using the `File` class:

```
File('something.txt').writeAsString("hello");
File('another.abc').writeAsBytes([0x3, 0xA2]);
```

When creating a `File` object you can also decide the mode, which can be one of the following:

- `FileMode.append`: Read and write at the end of the file (which is created if it doesn't exist)
- `FileMode.read`: File can only be read
- `FileMode.write` (default): Used to read and write the file (which is overwritten if it already exists or created if it doesn't exist).
- `FileMode.writeOnly`: the same as `write` but there's no possibility to read the contents.

By default `File` is in `write` mode but if you don't want, for example, overriding the file every time, just create it with the `append` mode.

A.1.2 Directories

Working with directories is quite easy thanks to `class Directory` which exposes many useful methods. An object of type `Directory` must contain a path on which operations are executed and it can be absolute or relative.

> ℹ In Flutter you should avoid using this class directly. Instead, prefer using the *path_provider* package which can be seen as the cross-platform version of `Directory`.

It automatically resolves the correct paths to the various system folders.

Like it happened with files, most of the operations are asynchronous (or streams) so that the main application is not blocked by I/O operations.

```
final dir = await Directory('folder1/folder2')
    .create(recursive: true);
```

Thanks to `recursive: true` you're sure that the target directory (*folder2*) is created **together** with its parents (*folder1*) if they don't exist. There are a series of "classic" methods you'd expect to be able to use on this kind of object:

- `createTemp()`: creates a temporary directory in the given directory;
- `rename()`: renames a directory;
- `exists()`: used to determine whether a directory exists or not;
- `delete()`: deletes a directory;
- `list()`: lists all the sub-directories and files

There's also the possibility to use a `Stream<FileSystemEvent> watch()` to listen for actions happening in the directory such as insertions or deletions.

A.1.3 Server side Dart

With Dart you can easily setup an HTTP server using `HttpServer`, which heavily relies on `Future<T>` and `Stream<T>`, just by providing an IP address and a specific port. Thanks to its asynchronous nature, it can listen for requests and handle more than one at the same time:

```
import 'dart:io';

Future<void> main() async {
    // Start server on localhost:8075
    var server = await HttpServer.bind(
        InternetAddress.loopbackIPv4, 8075
    );

    // Asynchronously handle incoming requests
    await for (HttpRequest request in server) {
        request.response.write("""<html><body>
```

```
                <p>Hello client!</p>
            </body></html>""");

            await request.response.close();
        }
}
```

The class `HttpRequest` is a server-side object containing information about the received HTTP request. Of course you can have more control on the incoming requests but the approach is quite "low level" as you manually have to handle the entire response lifecylce. This is a more elaborate example:

```
void main() {
    var server = await HttpServer.bind(
        InternetAddress.loopbackIPv4, 8075
    );

    await for (HttpRequest request in server) {
        handle(request);
    }
}

void handle(HttpRequest request) {
    if (request.method == 'GET') {
        handleGet(request);
    } else {
        if (request.method == 'POST') {
            handlePost(request);
        } else {
            request.response
                ..statusCode = HttpStatus.methodNotAllowed
                ..write('${request.method} not handled')
                ..close();
        }
    }
}
```

You can also run an HTTP**S** server using `bindSecure()` rather than `bind()`. For a complete

tutorial about writing HTTP servers with Dart, refer to the official documentation [1]. You can find on https://pub.dev a very convenient package called *http_server*, a higher level way of building HTTP servers with Dart. It's easier to use:

```
Future<void> main() async {
    var htmlPages = VirtualDirectory('www_root');

    htmlPages.directoryHandler = (directory, request) {
        final homePath = Uri.file(directory.path).resolve('index.html');
        final homePage = File(homePath.toFilePath());

        htmlPages.serveFile(homePage, request);
    };

    final server = await HttpServer.bind(
        InternetAddress.loopbackIPv4, 8075
    );

    await server.forEach(htmlPages.serveRequest);
}
```

The `VirtualDirectory` class is a secure way to serve files and directories in response to HTTP requests. It can be used to return HTML pages together with mime-types, potential error codes and so on. Be sure to check out the official package documentation [2] to get more info.

A.2 Date and time

In Dart the `Duration` class is used to represent time spans and making conversions from one unit to another. It's used to represent the difference between two moments, which can be negative in case the difference were from a later time to an earlier one.

```
// It also has 'days', 'milliseconds' and 'microseconds'
final myWorkToday = Duration(
    hours: 8,
    minutes: 25,
    seconds: 38,
);
```

[1] https://dart.dev/tutorials/server/httpserver
[2] https://pub.dev/documentation/http_server/latest/http_server/http_server-library.html

The above time span can be converted into seconds, for example, using the `inSeconds` getter which will return the **total** converted value of the interval. Look at this example to better understand how it works:

```
final total = Duration(
    hours: 9,
    minutes: 11,
    seconds: 37
);

// 9 hours, 11 minutes and 37 seconds are equivalent to 33097 TOTAL seconds:
// 9 * 3600 + 11 * 60 + 37 = 33097
print("${total.inSeconds}"); // prints 33097

// Returns the remainder of the division of the duration by 60. In fact
// 33097 % 60 = 37
print("${total.inSeconds.remainder(60)}"); // prints 37
```

In practice, `inSeconds` returns the representation of the **entire** duration in seconds. If you want to only extract the `seconds` parameter of `Duration`, you need to make a division. The same gist also applies to other getters such as `inMinutes`.

```
final d1 = const Duration(minutes: 15);
final d2 = const Duration(seconds: 548);

final d3 = (d1 < d2) ? d1 + d2 : d1 - d2;
```

Thanks to operator overloads you can easily compare objects and add/subtract values. If you want to represent a point in the time, use the `DateTime` class instead, which is also part of `dart:core`. Instances are generally created with one of the following constructors:

```
final d1 = DateTime.now();
final d2 = DateTime(1997, 5, 20); // May 20, 1997
final d3 = DateTime.utc(1997, DateTime.may, 20); // uses the UTC timezone
final d4 = DateTime.parse("1997-05-20 15:29:10Z");
```

Unless the `utc()` named constructor is used, the object is tied to the current device's date and time. Use a `DateTime` in conjunction with the "intl" package [3] to localize date and times. As always, be sure to check out the official documentation for a complete overview of the numerous

[3] See chapter 13

available getters [4].

```
final today = new DateTime.now();
final tomorrow = today.add(const Duration(day: 1));
```

A `DateTime` object doesn't change once created and has no operator overloads. Use the `Stopwatch` class if you're looking for a precise tool to measure the elapsed time between one or more calls. The two main methods you'll almost always use are `start()` and `stop()`:

```
final sw = Stopwatch();
print("Startng...");

sw.start();
await executeSomething();
sw.stop();

print("Elapsed = ${sw.elapsedMilliseconds}");
print("Elapsed = ${sw.isRunning}"); // false
```

Enclose the parts of code you want to "benchmark" between `start()` and `stop()` to retrieve the elapsed milliseconds or microseconds. Note that `reset()` doesn't start or stop the timer: it just sets the counter back to 0.

```
sw.start();
await Future.delayed(const Duration(seconds: 2));
sw.reset();
await Future.delayed(const Duration(seconds: 1));
sw.stop();
print("Elapsed = ${sw.elapsedMilliseconds}"); // Elapsed = 1000

sw.start();
await Future.delayed(const Duration(seconds: 2));
await Future.delayed(const Duration(seconds: 1));
sw.stop();
print("Elapsed = ${sw.elapsedMilliseconds}"); // Elapsed = 3000
```

Be sure to call `reset()` whenever you want to reset the timer. If you wish to convert the results in other time units, just wrap the total microseconds (or milliseconds) in a `Duration` object.

```
// Measure
```

[4] https://api.dart.dev/stable/2.8.3/dart-core/DateTime-class.html

```
sw.start();
await Future.delayed(const Duration(
    seconds: 2,
    milliseconds: 75
));
sw.stop();

// Wrap the time in a convenient object
final duration = Duration(
    microseconds: sw.elapsedMicroseconds
);
print(duration);
```

A.3 Obfuscating Dart code

After you've published a new project to the official stores, someone could download your Flutter app's binary and use some tools to see the internals (including assets and source code). This process is always doable and it's called **reverse engineering**. You can use code obfuscation to make the binary harder for humans to understand.

> ℹ Note that code obfuscation does **not** encrypt your Dart files: it just makes them harder for a human to read. In this way, the attacker will have harder times trying to figure out the actual logic but data is still visible.

In order to obfuscate a Flutter app, you need to make a build in **release** mode and append the `--obfuscate` flag at the end of the command. In addition, there should be `--split-debug-info` to tell Flutter where output files has to be written.

```
flutter build appbundle --obfuscate --split-debug-info=/path/to/a/folder/
flutter build apk --obfuscate --split-debug-info=/path/to/a/folder/
flutter build ios --obfuscate --split-debug-info=/path/to/a/folder/
```

Files generated at the path specified by `--split-debug-info` are needed if you want to de-obfuscate the binary in a second moment. Keep them in a secure place. We strongly recommend you to obfuscate your binary before publishing it to the official stores.

> ℹ The usage of `--split-debug-info` could drastically **reduce** the code size so you should really use it. It also works without `--obfuscate` but you'd better go for obfuscation + splitting.

Code obfuscation doesn't work for web apps simply because there's no need for it (web apps are **minified**, not obfuscated).